5TH SEMESTER MINI-PROJECT | ACADEMIC DEFENSE GUIDE

ProblemHub — Complete Presentation & Defense Guide

Platform Architecture, RBAC Lifecycle, Live Demo Script, and Technical Q&A

1. The 30-Second Elevator Pitch

> *"Good morning respected faculty members and examiners. Every year, thousands of engineering students build repetitive, isolated projects like generic calculator apps or basic to-do lists that get graded and discarded. Meanwhile, real-world NGOs, startups, and local community bodies face genuine technical challenges but lack developer bandwidth.*
>
> *Our project, **ProblemHub**, bridges this exact gap. It is a collaborative academic project platform where authenticated industry bodies and faculty propose real-world problem statements, student teams match their skills to claim and develop solutions under faculty supervision, and completed projects are archived as working solutions with future scope for subsequent batches to build upon."*

2. Problem Statement & Motivation

Current Academic Reality	ProblemHub Solution
Repetitive & Trivial Topics: Batches repeat the same library/banking management systems year after year.	**Authentic Problem Ingestion**: Real challenges submitted directly by industry partners, research faculty, and NGOs.
No Accountability / Guidance: Progress is tracked on paper or last-minute logbook signings.	**Milestone-Driven Tracking**: Digital milestone updates (`PROPOSED` → `IN_PROGRESS` → `COMPLETED`) verified by faculty guides.
Siloed Solutions: Once evaluated, code is forgotten and never reused.	**Institutional Solution Repository**: Open-source solutions with GitHub repos, live demos, and documented "Future Scope" for junior batches.
Skill Mismatch: Students pick topics without knowing required domain technologies.	**Content-Based Recommendation Engine**: Matches student skill sets with problem tech stacks using domain tag matching.

3. High-Level Architecture & Tech Stack

The project strictly follows a transparent **3-Tier Layered Architecture** using 5th-semester core concepts:

PRESENTATION LAYER (VIEW)
Thymeleaf HTML5 Templates • Semantic CSS & Responsive Grid • Vanilla JS & Chart.js Visualizations

↓ HTTP Requests / Model Rendering ↓

CONTROLLER & SECURITY LAYER (SPRING WEB MVC)
`AuthController` • `ProblemController` • `ProjectController` • `SolutionController` • `AuthInterceptor` (RBAC Guard)

↓ Business Services & DTOs ↓

BUSINESS LOGIC LAYER (SERVICES)
`RecommendationService` (Explainable Content-Based Skill Matching) • `AnalyticsService` (Domain Aggregations)

↓ DAO Method Calls ↓

DATA ACCESS LAYER (DAO VIA PURE JDBC)
`UserDAO` • `ProblemDAO` • `ProjectDAO` • `SolutionDAO` • `PreparedStatement` & `try-with-resources`

↓ TCP Connection Pool (HikariCP) ↓

RELATIONAL STORAGE LAYER (MYSQL 8.0)
`problemhub_db` with 13 Normalized Tables, Foreign Key Constraints, and B-Tree Indexes

Why this Tech Stack?

- **Java 17 & Spring Boot 3**: Provides enterprise web container (Embedded Tomcat), session management, and MVC routing.
- **Thymeleaf SSR**: Server-Side Rendering compiles HTML on the server, avoiding SPA complexity and ensuring fast load times.
- **Raw JDBC with PreparedStatement (No JPA/Hibernate)**: Demonstrates direct mastery of relational schemas and SQL queries; 100% parameterization guarantees complete immunity to SQL Injection attacks.
- **MySQL 8.0 (InnoDB)**: Full ACID compliance with relational constraints preventing orphaned data.

4. Role-Based Access Control (RBAC) & Lifecycle

🎓 **Student**

Discovers matched challenges, **proposes project solutions**, and updates milestones (10%–100%). Cannot post problem statements.

👨‍🏫 **Faculty Guide**

Publishes academic challenges, reviews & approves student proposals into `IN_PROGRESS`, and verifies final deliverables.

🏢 **Industry / NGO**

Posts authentic organizational bottlenecks, community issues, sets required technical domains, and reviews past solutions.

🛡️ **Administrator**

System governance, platform auditing, moderating reported challenges, and inspecting global analytics.

The Project Approval & Completion Pipeline

State	Progress	Description	Actor Authorized
`PROPOSED`	10%	Initial student project proposal with team details and abstract.	Student Submits
`IN_PROGRESS`	25% – 75%	Faculty guide reviews and approves project; team submits milestones.	Faculty Approves
`UNDER_REVIEW`	90%	Final code repository, video demo, and documentation submitted.	Student Submits
`COMPLETED`	100%	Faculty verifies all deliverables and releases project into Solutions Archive.	Faculty Verifies
`ARCHIVED`	Archive	Solution preserved with GitHub URL and "Future Scope" for subsequent batches.	Public Access

5. Step-by-Step Live Demo Script

STEP 1

The Landing Page & Ecosystem Overview (http://localhost:8080/)

> **What to say:** "Here is our homepage. It displays the ProblemHub workflow, live database metrics (active problems, student projects, archived solutions, and organizations), and recent problem statements with technologies and difficulty levels."

STEP 2

Problem Discovery & Search (http://localhost:8080/problems)

> **What to say:** "Students can filter problems by domain, difficulty, and organization. Notice that as an unauthenticated or student user, there is no option to create problems — problems originate solely from verified partner organizations or faculty."

STEP 3

Student Flow — Proposing a Project (Demo Login: Rahul Sharma)

> **What to say:** "When logged in as student Rahul Sharma, his dashboard shows his claimed skills (Java, Spring Boot, MySQL), active projects, and intelligent recommendations. When Rahul proposes a project, the system initializes it in the `PROPOSED` state at 10% milestone awaiting faculty approval."

STEP 4

Faculty Flow — Reviewing & Approving (Demo Login: Dr. Arvind Kulkarni)

> **What to say:** "Switching to the Faculty view, Dr. Kulkarni has an exclusive dashboard where he can review all pending student proposals, approve them to move into `IN_PROGRESS`, and mentor teams through their milestone submissions."

STEP 5

Solution Repository & Future Scope (http://localhost:8080/solutions)

> **What to say:** "Once completed, projects enter the Solutions Archive. Each record includes verified GitHub repositories, live demo links, and crucial 'Future Scope' notes, preventing next year's juniors from reinventing the wheel."

STEP 6

Live Analytics & Interactive REST API Docs (http://localhost:8080/api-docs)

> **What to say:** "We incorporated real-time domain distribution metrics using Chart.js, and documented our REST API endpoints (`/api/problems`, `/api/solutions`, `/api/recommendations`) for multi-platform integration."

6. Key Code Highlights in IDE

Class / File	Key Concepts to Point Out to Faculty
`AuthInterceptor.java`	Implements `HandlerInterceptor`. Blocks students from `/problems/new`. Verifies session authentication and enforces role boundaries.
`ProblemDao.java`	Uses `PreparedStatement` with parameter placeholders (`?`), explicit `ResultSet` mapping, and automatic resource cleanup via `try-with-resources`.
`ProjectController.java`	Enforces IDOR ownership protection (`project.getStudentUserId() == currentUser.getId()`) on progress updates and initializes new proposals as `PROPOSED` (10%).
`RecommendationService.java`	Pure Java explainable Content-Based Filtering using Jaccard Similarity on technical domain tags and student skills. No black-box dependencies.
`ProblemHubApplicationTests.java`	10 automated unit test cases validating role-based authorization, initial proposed status, anti-self-rating, and security interceptors (100% build pass).

7. Technical Q&A Defense (Faculty Questions & Rebuttals)

Q1: "Why did you use raw JDBC instead of Hibernate or Spring Data JPA?"

Answer: "In academic environments, heavy ORMs like Hibernate often hide SQL behind automated reflection, leading to the N+1 select query problem and high memory overhead. By implementing raw JDBC with `PreparedStatement`, we demonstrate full control over relational indexing, explicit SQL execution, and guaranteed zero connection leaks using `try-with-resources`."

Q2: "Can a student bypass the UI and post a problem using Postman or curl?"

Answer: "No, sir. We implemented server-side defense-in-depth. In both `AuthInterceptor.java` and `ProblemController.java`, the server inspects the session user role on `POST /problems/post`. If the user has the `STUDENT` role, the request is immediately rejected and redirected with an `unauthorized_post_problem` security flag."

Q3: "What prevents a student from tampering with another team's project progress?"

Answer: "In `ProjectController.java`, the progress update endpoint (`POST /projects/{id}/update-progress`) queries the database and verifies that `project.getStudentUserId() == currentUser.getId()`. Unless the user is the project owner, faculty guide, or admin, the update is blocked, preventing Insecure Direct Object References (IDOR)."

Q4: "How does your recommendation algorithm work? Is it AI?"

Answer: "It is an explainable Content-Based Filtering algorithm written in pure Java. It calculates the Jaccard similarity index between the student's claimed technical skill tags and the problem statement's required domain tags:
`Similarity = |StudentSkills ∩ ProblemTags| / |StudentSkills ∪ ProblemTags|`. It ranks recommendations transparently without unexplainable AI models."

Q5: "How is database consistency maintained under simultaneous user requests?"

Answer: "We use MySQL's InnoDB storage engine, which provides full ACID compliance with row-level locking. Relational integrity is enforced using foreign key constraints with `ON DELETE RESTRICT` to prevent orphaned child records."

8. Suggested 10-Minute Presentation Split

Time	Agenda Topic	Key Points to Cover
0:00 – 1:30	Introduction & Academic Motivation	Elevator pitch, stopping repetitive projects, authentic industry/NGO intake.
1:30 – 3:30	System Architecture & Database	3-Tier model, JDBC PreparedStatements, 13 normalized tables.
3:30 – 7:30	Live Platform Demonstration	Explore → Student Proposes → Faculty Approves → Solution Archive.
7:30 – 8:30	Security & Design Decisions	RBAC role guards, IDOR prevention, explainable recommendation logic.
8:30 – 10:00	Conclusion & Faculty Q&A	Future scope, testing results (10/10 tests pass), addressing faculty questions.

9. Pre-Presentation Checklist

- **Database Running**: Verify MySQL is active on port 3306 (`mysql –u root –p problemhub_db`).
- **Server Started**: Run `./mvnw spring–boot:run` and verify [http://localhost:8080/] (http://localhost:8080/) returns HTTP 200.
- **Two Browser Windows Ready**: One standard window for Student (Rahul Sharma) and one Incognito window for Faculty (Dr. Arvind Kulkarni) to demonstrate real-time interactions without re-logging in.